Differential Flatness Control of a PMSM

A Tutorial Derivation from the Mechanical Flatness Output to Jerk-Limited Trajectory Generation and Voltage Feedforward

EmbedSim EV Light Vehicle Foundation

August 2026

Abstract

This paper presents a tutorial derivation of the Differential Flatness Control (DFC) strategy implemented in the EmbedSim PMSM controller. The purpose is not only to state the final control equations, but to show how they arise from the PMSM differential equations. Particular attention is given to the meaning of Lie derivatives, the selection of a flat output, the construction of a reference trajectory, and the conversion of the desired mechanical motion into torque, current, and voltage feedforward commands.

The implementation uses a jerk-limited velocity trajectory. The desired velocity, acceleration, and jerk are then used by the flatness equations to construct the current and voltage references. A conventional speed PI loop is retained as a feedback correction, while inner current PI loops compensate for modelling errors and disturbances. Thus the practical architecture is

S-curve trajectory → DFC feedforward → speed PI correction → current PI → SVPWM.

The derivation is presented first for a surface PMSM with $L_d = L_q = L$, because this is the form directly represented by the implemented feedforward equations. The extension to a salient PMSM is discussed later.

Contents

1 Introduction

Differential flatness provides a systematic way of constructing feedforward control laws from a small number of physically meaningful output variables. For a PMSM, the central idea is particularly attractive because the desired mechanical motion can be converted analytically into the torque, current and voltage required to produce that motion.

A conventional field-oriented controller is normally written as a cascade: a speed controller produces a torque or current reference, and current controllers produce voltage commands. Differential flatness adds an important second path: instead of asking the feedback controller to generate all of the required torque, the desired acceleration is used directly in the mechanical equation.

The resulting decomposition is

$$T_e = \underbrace{J\dot{\omega}_r + B\omega_r + T_L}_{\text{model-based feedforward}} + \underbrace{\Delta T}_{\text{feedback correction}} .$$

The first term is obtained directly from the mechanical model. The second term is supplied by a conventional speed PI controller.

The EmbedSim implementation follows this principle. The controller generates a smooth jerk-limited trajectory and obtains

$$\omega^\star, \qquad \dot{\omega}^\star, \qquad \ddot{\omega}^\star.$$

These quantities are then used to calculate torque, i_q, $\dot{i}_q$ and finally v_d and v_q. The implementation also transforms measured phase currents into the dq reference frame and closes the current feedback loops.

The source implementation explicitly describes the controller as "differential-flatness feedforward with speed and current PI loops" and uses a smooth reference trajectory before the DFC calculation. The control module targets embedded 32-bit microcontrollers such as Infineon AURIX TriCore and ARM Cortex-M4. [1]

2 The PMSM Model

2.1 Mechanical equation

Let

$$\theta_m$$

be the mechanical rotor angle and

$$\omega_m = \dot{\theta}_m$$

the mechanical angular velocity.

The mechanical dynamics are

$$J\dot{\omega}_m = T_e - B\omega_m - T_L, \tag{1}$$

where

J : rotor and load inertia,
B : viscous friction coefficient,
T_e : electromagnetic torque,
T_L : external load torque.

[1] See the supplied EmbedSim source documentation: `embed_sim_dfc_controller.c` and `embed_sim_control.c`.

Solving (1) for the electromagnetic torque gives

$$T_e = J\dot{\omega}_m + B\omega_m + T_L. \tag{2}$$

This is already the essential mechanical flatness relation.

If the desired velocity is $\omega^\star(t)$, then the torque required to follow that velocity exactly is

$$T_e^\star = J\dot{\omega}^\star + B\omega^\star + T_L. \tag{3}$$

The controller therefore does not need to "discover" the required torque through the speed PI loop. The desired acceleration already tells us how much torque is required.

3 Electrical PMSM Equations

In rotor-oriented dq coordinates, the general PMSM voltage equations are

$$v_d = R_s i_d + L_d \dot{i}_d - \omega_e L_q i_q, \tag{4}$$

$$v_q = R_s i_q + L_q \dot{i}_q + \omega_e (L_d i_d + \lambda_{pm}), \tag{5}$$

where

$$\omega_e = p\omega_m$$

is the electrical angular velocity and p is the number of pole pairs.

For a surface PMSM, or for the simplified model used by the feedforward implementation,

$$L_d = L_q = L.$$

With $i_d = 0$,

$$v_d = -\omega_e L i_q, \tag{6}$$

$$v_q = R_s i_q + L\dot{i}_q + \omega_e \lambda_{pm}. \tag{7}$$

The implemented equations have exactly this structure, using L_q in the $L_d = L_q$ case:

$$v_d^\star = -p\omega^\star L_q i_q^\star,$$

and

$$v_q^\star = R_s i_q^\star + L_q \dot{i}_q^\star + p\omega^\star \lambda_{pm}.$$

4 Electromagnetic Torque

The general PMSM torque equation is

$$T_e = \frac{3}{2} p \left[\lambda_{pm} i_q + (L_d - L_q) i_d i_q\right]. \tag{8}$$

For a surface PMSM with

$$L_d = L_q,$$

the reluctance torque disappears and

$$T_e = \frac{3}{2} p \lambda_{pm} i_q. \tag{9}$$

Define the torque constant

$$K_t = \frac{3}{2} p \lambda_{pm}. \tag{10}$$

Then

$$T_e = K_t i_q. \tag{11}$$

Therefore the q-axis current required for a desired torque is simply

$$i_q^\star = \frac{T_e^\star}{K_t}. \tag{12}$$

Substituting (3),

$$\boxed{i_q^\star = \frac{J\dot{\omega}^\star + B\omega^\star + T_L}{K_t}}. \tag{13}$$

This is the first important flatness equation.

5 What Does Differential Flatness Mean?

The word "flat" can initially be confusing. It does not mean that the motor has no dynamics. It means that the complete system state and the control inputs can be expressed using a finite number of derivatives of specially selected outputs, called *flat outputs*.

Consider a nonlinear system

$$\dot{x} = f(x) + \sum_{j=1}^{m} g_j(x)u_j, \qquad y = h(x), \tag{14}$$

where

$$x \in \mathbb{R}^n, \qquad u \in \mathbb{R}^m.$$

The system is differentially flat if there exists an output

$$y$$

such that

$$x = \Phi_x(y, \dot{y}, \ddot{y}, \ldots, y^{(r)}),$$

and

$$u = \Phi_u(y, \dot{y}, \ddot{y}, \ldots, y^{(r+1)}).$$

Thus, instead of solving the nonlinear state equations forward to find the control input, we can specify the desired output trajectory first and then calculate the required input.

For motor control this is extremely useful:

$$\boxed{\text{desired motion} \rightarrow \text{required torque} \rightarrow \text{required current} \rightarrow \text{required voltage.}}$$

6 Lie Derivatives: The Basic Idea

Lie derivatives provide the mathematical language used to determine how an output changes along the trajectories of a nonlinear dynamical system.

Suppose

$$\dot{x} = f(x)$$

and

$$y = h(x).$$

The first Lie derivative of h along f is

$$L_f h(x) = \nabla h(x)\, f(x). \tag{15}$$

Because

$$y = h(x),$$

we have

$$\dot{y} = \nabla h(x)\dot{x} = \nabla h(x) f(x) = L_f h(x). \tag{16}$$

The second Lie derivative is

$$L_f^2 h = L_f(L_f h), \tag{17}$$

and therefore

$$\ddot{y} = L_f^2 h. \tag{18}$$

Repeatedly differentiating the output gives

$$y^{(r)} = L_f^r h. \tag{19}$$

For a control-affine system,

$$\dot{x} = f(x) + g(x)u,$$

the first derivative is

$$\dot{y} = L_f h + L_g h\, u. \tag{20}$$

The input appears when the Lie derivative

$$L_g L_f^{r-1} h$$

becomes nonzero.

This is the mathematical reason that repeated output differentiation can reveal the control input.

7 Lie Derivatives Applied to the PMSM

For clarity, consider the simplified state

$$x = \begin{bmatrix} \theta_m \\ \omega_m \\ i_q \end{bmatrix}.$$

For a surface PMSM with $i_d = 0$ and constant load torque, the equations are

$$\dot{\theta}_m = \omega_m, \tag{21}$$

$$\dot{\omega}_m = \frac{1}{J}\left(K_t i_q - B\omega_m - T_L\right), \tag{22}$$

$$\dot{i}_q = \frac{1}{L}\left(v_q - R_s i_q - p\omega_m \lambda_{pm}\right). \tag{23}$$

Choose the mechanical angle as output:

$$y = \theta_m. \tag{24}$$

Then

$$\dot{y} = \omega_m. \tag{25}$$

Differentiate again:

$$\ddot{y} = \dot{\omega}_m \tag{26}$$

$$= \frac{1}{J}\left(K_t i_q - B\omega_m - T_L\right). \tag{27}$$

The current is therefore reconstructed directly from the output and its second derivative:

$$\boxed{i_q = \frac{J\ddot{y} + B\dot{y} + T_L}{K_t}.} \tag{28}$$

Differentiate once more:

$$\dddot{y} = \frac{1}{J}\left(K_t \dot{i}_q - B\dot{\omega}_m\right). \tag{29}$$

Therefore

$$\dot{i}_q = \frac{J\,\dddot{y} + B\ddot{y}}{K_t}, \tag{30}$$

assuming constant T_L.

Now use the electrical equation:

$$v_q = R_s i_q + L\dot{i}_q + p\omega_m \lambda_{pm}.$$

Substitution gives

$$\boxed{v_q = R_s i_q + L\frac{J\,\dddot{y} + B\ddot{y}}{K_t} + p\dot{y}\lambda_{pm}.} \tag{31}$$

Similarly,

$$v_d = -p\dot{y}L i_q. \tag{32}$$

This demonstrates the essential flatness property:

$$\boxed{\theta_m, \dot{\theta}_m, \ddot{\theta}_m, \dddot{\theta}_m \quad \Longrightarrow \quad i_q, \dot{i}_q, v_d, v_q.}$$

The input voltages can therefore be constructed from the desired output trajectory and a finite number of its derivatives.

8 Why the Implementation Uses Speed as the Trajectory Variable

Although the mathematical flat output can be selected as rotor position, the implementation naturally works with velocity.

Define

$$\omega^\star(t)$$

as the desired mechanical speed trajectory.

Then

$$\dot{\omega}^\star(t)$$

is the desired acceleration and

$$\ddot{\omega}^\star(t)$$

is the desired jerk.

The mechanical feedforward equation becomes

$$T_e^\star = J\dot{\omega}^\star + B\omega^\star + T_L. \tag{33}$$

Consequently,

$$\boxed{i_q^\star = \frac{J\dot{\omega}^\star + B\omega^\star + T_L}{K_t}.} \tag{34}$$

Differentiating,

$$\dot{i}_q^\star = \frac{J\ddot{\omega}^\star + B\dot{\omega}^\star + \dot{T}_L}{K_t}. \tag{35}$$

For a constant or slowly varying load,

$$\dot{T}_L \approx 0,$$

and therefore

$$\boxed{\dot{i}_q^\star = \frac{J\ddot{\omega}^\star + B\dot{\omega}^\star}{K_t}.} \tag{36}$$

This is precisely the structure used by the supplied DFC implementation.

9 From the Reference Trajectory to Voltage Feedforward

Once

$$\omega^\star, \quad \dot{\omega}^\star, \quad \ddot{\omega}^\star$$

are known, the complete feedforward chain is straightforward.

9.1 Step 1: Torque

$$T_{ff} = J\dot{\omega}^\star + B\omega^\star + T_L. \tag{37}$$

9.2 Step 2: Feedback torque correction

The speed error is

$$e_\omega = \omega^\star - \omega_m. \tag{38}$$

A PI controller produces

$$\Delta T = K_{p,\omega} e_\omega + K_{i,\omega} \int e_\omega \, dt. \tag{39}$$

The total torque request is therefore

$$\boxed{T^\star = T_{ff} + \Delta T.} \tag{40}$$

This is an important practical point: the DFC controller is not isolated from the measured motor state. The model generates the main feedforward action, while the speed feedback corrects model error and disturbances.

9.3 Step 3: Current reference

$$i_q^\star = \frac{T^\star}{K_t}. \tag{41}$$

9.4 Step 4: Current derivative

The feedforward derivative uses the trajectory terms:

$$\dot{i}_q^\star = \frac{J\ddot{\omega}^\star + B\dot{\omega}^\star}{K_t}. \tag{42}$$

9.5 Step 5: Voltage reference

The feedforward voltage is

$$v_d^\star = -p\omega^\star L i_q^\star, \tag{43}$$

$$v_q^\star = R_s i_q^\star + L\dot{i}_q^\star + p\omega^\star \lambda_{pm}. \tag{44}$$

Thus

$$\boxed{\begin{bmatrix} v_d^\star \\ v_q^\star \end{bmatrix} = \begin{bmatrix} -p\omega^\star L i_q^\star \\ R_s i_q^\star + L\dot{i}_q^\star + p\omega^\star \lambda_{pm} \end{bmatrix}.} \tag{45}$$

10 Why a Jerk-Limited S-Curve Is Used

A speed step is mathematically possible but physically undesirable. A discontinuous speed reference produces an extremely large acceleration. A discontinuous acceleration produces an impulse-like jerk.

The trajectory generator therefore limits

$$|\dot{\omega}| \leq a_{\max}$$

and

$$|\ddot{\omega}| \leq j_{\max},$$

where the notation here follows the implementation: velocity ω, acceleration $\dot{\omega}$ and jerk $\ddot{\omega}$.

The resulting trajectory is smooth:

$$\omega(t) \quad \text{is continuous,}$$

$$\dot{\omega}(t) \quad \text{is bounded,}$$

and

$$\ddot{\omega}(t) \quad \text{is bounded.}$$

This is particularly valuable for DFC because the feedforward controller explicitly uses acceleration and jerk.

11 Derivation of the Stopping Acceleration

The trajectory generator needs to know how strongly it should accelerate or decelerate in order to reach the target without overshooting.

Let

$$e_\omega = \omega^\star - \omega.$$

Assume a constant maximum jerk magnitude

$$j_{\max} > 0.$$

Suppose an acceleration of magnitude a_s is required to remove the remaining velocity error. For a constant jerk deceleration from a_s to zero, the velocity change associated with the triangular acceleration profile is

$$\Delta\omega = \frac{a_s^2}{2j_{\max}}. \tag{46}$$

Solving for a_s gives

$$a_s = \sqrt{2j_{\max}|\Delta\omega|}. \tag{47}$$

Hence

$$\boxed{a_{\text{stop}} = \sqrt{2j_{\max}|e_\omega|}.} \tag{48}$$

The implementation uses exactly this relation:

$$a_{\text{stop}} = \sqrt{2j_{\max}|e_\omega|}.$$

The desired acceleration is then selected in the direction of the velocity error and limited by the maximum acceleration:

$$a^\star = \operatorname{sgn}(e_\omega)\min\left(a_{\max}, a_{\text{stop}}\right). \tag{49}$$

This is the key equation behind the online jerk-limited trajectory generator.

12 Discrete-Time Path Generation

Let the controller sample time be

$$T_s.$$

At sample k, define

$$\omega_k, \qquad a_k, \qquad j_k.$$

The requested jerk needed to move the acceleration toward the desired value is

$$j_{\mathrm{req},k} = \frac{a_k^\star - a_k}{T_s}. \tag{50}$$

The actual jerk is saturated:

$$\boxed{j_k = \mathrm{sat}\left(j_{\mathrm{req},k}, -j_{\max}, +j_{\max}\right).} \tag{51}$$

Assuming constant jerk during one sample, integrate acceleration:

$$a_{k+1} = a_k + j_k T_s. \tag{52}$$

Integrate velocity:

$$\boxed{\omega_{k+1} = \omega_k + a_k T_s + \frac{1}{2} j_k T_s^2.} \tag{53}$$

Integrate position:

$$\boxed{\theta_{k+1} = \theta_k + \omega_k T_s + \frac{1}{2} a_k T_s^2 + \frac{1}{6} j_k T_s^3.} \tag{54}$$

These equations are not merely numerical approximations of arbitrary order. They follow directly from the exact kinematics of constant jerk over one sample.

The supplied implementation uses these same expressions for velocity, acceleration and position.

13 Complete DFC Algorithm

The resulting algorithm can be summarized as follows.

Step 1: Read the requested target speed

$$\omega_{\mathrm{target}}.$$

Step 2: Calculate the velocity error

$$e_\omega = \omega_{\mathrm{target}} - \omega_{\mathrm{ref}}.$$

Step 3: Calculate the stopping acceleration

$$a_{\mathrm{stop}} = \sqrt{2 j_{\max} |e_\omega|}.$$

Step 4: Limit the desired acceleration:

$$a^\star = \mathrm{sgn}(e_\omega) \min(a_{\max}, a_{\mathrm{stop}}).$$

Step 5: Calculate the requested jerk:

$$j_{\mathrm{req}} = \frac{a^\star - a}{T_s}.$$

Step 6: Saturate the jerk:

$$j = \mathrm{sat}(j_{\mathrm{req}}, -j_{\max}, j_{\max}).$$

Step 7: Integrate acceleration:

$$a^{+} = a + jT_s.$$

Step 8: Integrate velocity:

$$\omega^{+} = \omega + aT_s + \frac{1}{2}jT_s^2.$$

Step 9: Calculate the mechanical feedforward torque:

$$T_{ff} = Ja^{+} + B\omega^{+} + T_L.$$

Step 10: Add speed PI correction:

$$T^{\star} = T_{ff} + \Delta T.$$

Step 11: Convert torque into q-axis current:

$$i_q^{\star} = \frac{T^{\star}}{K_t}.$$

Step 12: Calculate current derivative feedforward:

$$\dot{i}_q^{\star} = \frac{Jj + Ba}{K_t}.$$

Step 13: Calculate the voltage feedforward:

$$v_d^{\star} = -p\omega^{\star} L i_q^{\star},$$

$$v_q^{\star} = R_s i_q^{\star} + L\dot{i}_q^{\star} + p\omega^{\star}\lambda_{pm}.$$

Step 14: Measure phase currents and transform them into dq:

$$(i_u, i_v, i_w) \to (i_\alpha, i_\beta) \to (i_d, i_q).$$

Step 15: Calculate current errors:

$$e_d = i_d^{\star} - i_d, \qquad e_q = i_q^{\star} - i_q.$$

Step 16: Apply current PI corrections:

$$v_d = v_d^{\star} + \Delta v_d,$$

$$v_q = v_q^{\star} + \Delta v_q.$$

Step 17: Apply inverse Park transformation and SVPWM.

14 Relationship to the Supplied EmbedSim Code

The supplied source implements the architecture described above.

The DFC controller first generates a smooth jerk-limited trajectory and reads the resulting reference velocity, acceleration and jerk. It then calculates the speed error and speed PI correction, followed by the mechanical feedforward torque:

$$T_{ff} = J\dot{\omega}^\star + B\omega^\star + T_L.$$

The code then converts torque to $i_q^\star$ through

$$K_t = 1.5\, p\, \lambda_{pm},$$

and computes

$$\dot{i}_q^\star = \frac{J\ddot{\omega}^\star + B\dot{\omega}^\star}{K_t}.$$

The voltage feedforward equations are

$$v_d^\star = -p\omega^\star L_q i_q^\star,$$

and

$$v_q^\star = R_s i_q^\star + L_q \dot{i}_q^\star + p\omega^\star \lambda_{pm}.$$

The measured phase currents are converted to dq coordinates and current PI corrections are added to the feedforward voltage. Finally, the voltage vector is transformed back and supplied to the SVM.

This mapping is important because it shows that the DFC equations are not a separate theoretical exercise: they correspond directly to the implemented control chain.

15 Where the Sensors Enter the "Model-Based" Controller

A common misunderstanding is that differential flatness means the controller is completely isolated from reality.

That is not the case.

The supplied implementation obtains rotor position and speed from the sensor interface/observer. The observer module copies the validated rotor position and speed into the controller state. During closed-loop operation it computes the electrical rotor angle and compares it with the internal SVM angle.

The angle error is

$$e_\theta = \mathrm{wrap}_{[-\pi,\pi)}\left(\theta_{e,\mathrm{sensor}} - \theta_{e,\mathrm{model}}\right). \tag{55}$$

The internal angle is corrected according to

$$\theta_{e,\mathrm{model}} \leftarrow \theta_{e,\mathrm{model}} + k_\theta e_\theta. \tag{56}$$

A speed-based feedforward term is additionally used for measurement-delay compensation.

Therefore the practical controller has two different concepts:

1. **Trajectory model:** predicts what the motor should do in order to follow the requested motion.
2. **Feedback/observer path:** measures what the real motor actually does and corrects the internal model and control errors.

This distinction is fundamental to understanding the architecture.

16 Why the Speed PI Is Still Necessary

The ideal flatness equation assumes that the parameters

$$J, \quad B, \quad T_L, \quad R_s, \quad L, \quad \lambda_{pm}$$

are known accurately.

In a real traction motor they are not perfectly known.

For example,

$$J = J_{\text{rotor}} + J_{\text{vehicle}}$$

can change with vehicle load.

Similarly, friction can be nonlinear, and the load torque can change rapidly.

If the estimated inertia is

$$\hat{J} = J + \Delta J,$$

the feedforward torque becomes

$$T_{ff} = \hat{J}\dot{\omega}^\star + B\omega^\star + T_L.$$

The error

$$\Delta T = (J - \hat{J})\dot{\omega}^\star$$

cannot be eliminated by the model alone.

The speed PI loop provides exactly this missing robustness:

$$T^\star = T_{ff} + \Delta T.$$

Thus DFC and PI are complementary rather than competing methods.

17 Current Feedback Layer

The feedforward current is

$$i_q^\star.$$

The measured current is

$$i_q.$$

The error is

$$e_q = i_q^\star - i_q. \tag{57}$$

The current PI controller produces

$$\Delta v_q = K_{p,q} e_q + K_{i,q} \int e_q \, dt. \tag{58}$$

Likewise, with

$$i_d^\star = 0,$$

$$e_d = -i_d \tag{59}$$

and

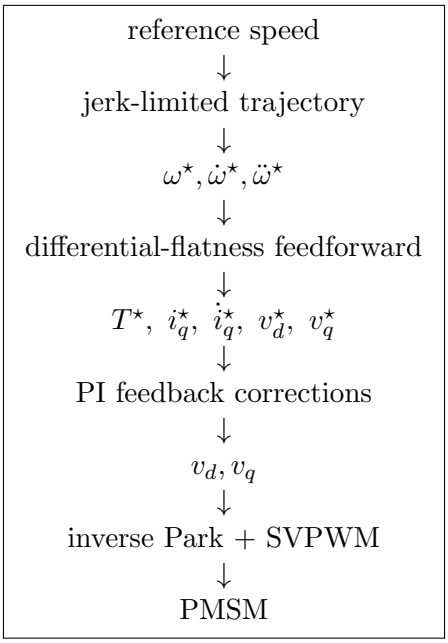
$$\Delta v_d = K_{p,d} e_d + K_{i,d} \int e_d \, dt. \tag{60}$$

The final commands are

$$v_d = v_d^\star + \Delta v_d, \tag{61}$$

$$v_q = v_q^\star + \Delta v_q. \tag{62}$$

This gives the final architecture:



18 Salient PMSM Extension

The derivation above assumes

$$L_d = L_q.$$

For a salient PMSM,

$$L_d \neq L_q$$

and the torque becomes

$$T_e = \frac{3}{2} p \left[\lambda_{pm} i_q + (L_d - L_q) i_d i_q \right]. \tag{63}$$

The electrical equations are

$$v_d = R_s i_d + L_d \dot{i}_d - \omega_e L_q i_q, \tag{64}$$

$$v_q = R_s i_q + L_q \dot{i}_q + \omega_e (L_d i_d + \lambda_{pm}). \tag{65}$$

If $i_d = 0$ is imposed, the reluctance torque disappears and the previous surface-PMSM result is recovered.

If nonzero i_d is deliberately used, the flatness mapping becomes more complicated because the torque depends on both currents. In that case the trajectory-to-current transformation must solve

$$T^\star = \frac{3}{2} p \left[\lambda_{pm} i_q^\star + (L_d - L_q) i_d^\star i_q^\star \right].$$

The current trajectory must then include both

$$i_d^\star, \qquad i_q^\star$$

and their derivatives.

For the present EmbedSim implementation, the implemented feedforward law is therefore most naturally interpreted as the $i_d = 0$ / non-salient form.

19 Important Implementation Detail: Units

The derivation is simplest when all mechanical angular quantities are in SI units:

$$\omega \text{ [rad/s]}, \qquad \dot{\omega} \text{ [rad/s}^2\text{]}, \qquad \ddot{\omega} \text{ [rad/s}^3\text{]}.$$

The source converts RPM references into rad/s before the DFC calculation.

The conversion is

$$\omega_{\text{rad/s}} = \omega_{\text{RPM}} \frac{2\pi}{60}. \tag{66}$$

This is particularly important for jerk:

$$j_{\text{rad/s}^3} = j_{\text{RPM/s}^2} \frac{2\pi}{60}.$$

A unit mistake here is amplified by the DFC equations because jerk appears inside the current-derivative feedforward:

$$\dot{i}_q^\star = \frac{Jj + Ba}{K_t}.$$

20 Numerical Protection and Saturation

The practical implementation contains several protections.

The current reference is limited:

$$|i_q^\star| \le I_{\max}.$$

The current derivative is limited:

$$|\dot{i}_q^\star| \le \dot{I}_{q,\max}.$$

The speed trajectory is limited by

$$|\omega| \le \omega_{\max}$$

and

$$|a| \le a_{\max}.$$

The requested jerk is limited by

$$|j| \le j_{\max}.$$

The voltage magnitude is also constrained before SVPWM.

These limits are not part of the ideal differential-flatness theorem itself. They are engineering constraints required to make the theoretical control law safe and implementable on a finite-voltage, finite-current inverter.

21 Startup and Transition to DFC

The supplied controller uses an open-loop startup phase before closed-loop DFC. During startup an SVM voltage command is generated using an internally integrated electrical angle. After the startup period, the controller switches to closed-loop DFC and reinitializes the trajectory/controller states.

This transition is important because a flatness controller assumes a meaningful electrical angle and a valid current transformation.

The source documentation describes the startup phase and the subsequent closed-loop DFC mode explicitly.

A small source-level point should be checked before publication: the DFC source comment describes the startup duration as 0.3 s while the supplied macro is currently set to

$$\texttt{DFC\_STARTUP\_DURATION\_S} = 3.0\text{F}.$$

The documentation and implementation should be made consistent.

22 Conceptual Interpretation

The most useful way to understand the entire method is to view the controller as a chain of physical equations.

First, the trajectory generator answers:

$$\boxed{\text{How should the motor speed change?}}$$

It produces

$$\omega^\star, \quad a^\star, \quad j^\star.$$

Second, the mechanical equation answers:

$$\boxed{\text{How much torque is required?}}$$

$$T^\star = Ja^\star + B\omega^\star + T_L + \Delta T.$$

Third, the torque equation answers:

$$\boxed{\text{How much } q\text{-axis current is required?}}$$

$$i_q^\star = \frac{T^\star}{K_t}.$$

Fourth, the electrical equation answers:

$$\boxed{\text{How much voltage is required?}}$$

$$v_d^\star = -p\omega^\star L i_q^\star,$$
$$v_q^\star = R_s i_q^\star + L\dot{i}_q^\star + p\omega^\star \lambda_{pm}.$$

Finally, the current sensors answer:

$$\boxed{\text{Did the real motor actually produce that current?}}$$

The PI loops then correct the difference.

This is why the DFC controller should not be viewed as a replacement for feedback control. It is better understood as a model-based feedforward generator embedded inside a feedback-controlled FOC architecture.

23 Final Derivation in One Chain

The complete derivation can be compressed into the following sequence.

1. Reference trajectory

$$\omega^\star, \qquad \dot{\omega}^\star, \qquad \ddot{\omega}^\star. \tag{67}$$

2. Mechanical equation

$$J\dot{\omega} = T_e - B\omega - T_L. \tag{68}$$

Therefore

$$T_{ff} = J\dot{\omega}^\star + B\omega^\star + T_L. \tag{69}$$

3. Speed feedback

$$e_\omega = \omega^\star - \omega, \tag{70}$$

$$\Delta T = K_{p,\omega} e_\omega + K_{i,\omega} \int e_\omega dt. \tag{71}$$

Hence

$$T^\star = T_{ff} + \Delta T. \tag{72}$$

4. Torque to current

$$K_t = \frac{3}{2} p \lambda_{pm}, \tag{73}$$

$$i_q^\star = \frac{T^\star}{K_t}. \tag{74}$$

5. Current derivative

$$\dot{i}_q^\star = \frac{J\ddot{\omega}^\star + B\dot{\omega}^\star}{K_t}. \tag{75}$$

6. Voltage feedforward

$$v_d^\star = -p\omega^\star L i_q^\star, \tag{76}$$

$$v_q^\star = R_s i_q^\star + L\dot{i}_q^\star + p\omega^\star \lambda_{pm}. \tag{77}$$

7. Current feedback

$$v_d = v_d^\star + \Delta v_d, \tag{78}$$

$$v_q = v_q^\star + \Delta v_q. \tag{79}$$

8. Modulation

$$(v_d, v_q) \to (v_\alpha, v_\beta) \to \text{SVPWM} \to (D_u, D_v, D_w).$$

Thus,

$$\boxed{\omega^\star \to (\dot{\omega}^\star, \ddot{\omega}^\star) \to T^\star \to i_q^\star \to \dot{i}_q^\star \to (v_d^\star, v_q^\star) \to \text{SVPWM}.} \tag{80}$$

24 Conclusion

Differential flatness gives a direct analytical bridge between desired mechanical motion and the electrical input required by the PMSM.

The key mathematical idea is that a suitable output and its derivatives contain enough information to reconstruct the internal state and the control inputs. Lie derivatives provide the formal language for this reconstruction. For the simplified PMSM considered here, choosing rotor position as the conceptual flat output leads naturally to the chain

$$\theta_m \to \omega_m \to \dot{\omega}_m \to \ddot{\omega}_m,$$

which in turn determines

$$i_q, \quad \dot{i}_q, \quad v_d, \quad v_q.$$

The practical EmbedSim implementation uses speed as the trajectory variable because it is the natural quantity specified by the application. A jerk-limited trajectory generator creates

a physically smooth reference. The mechanical equation converts acceleration into torque, the torque equation converts torque into q-axis current, and the electrical equations convert current and its derivative into voltage.

The speed PI and current PI loops then provide robustness against parameter uncertainty, load changes, measurement errors and inverter imperfections.

The resulting controller is therefore best described as

Jerk-limited trajectory + Differential-flatness feedforward + PI feedback + FOC/SVPWM.

This architecture retains the physical transparency of model-based control while preserving the robustness of conventional closed-loop FOC.

A Notation

Symbol	Meaning
θ_m	Mechanical rotor angle
ω_m	Mechanical angular velocity
ω_e	Electrical angular velocity
p	Number of pole pairs
J	Rotor/load inertia
B	Viscous friction coefficient
T_e	Electromagnetic torque
T_L	Load torque
R_s	Stator resistance
L_d, L_q	d- and q-axis inductances
λ_{pm}	Permanent-magnet flux linkage
i_d, i_q	dq currents
v_d, v_q	dq voltages
K_t	Torque constant $\frac{3}{2}p\lambda_{pm}$
T_s	Control sample time
a	Mechanical acceleration $\dot{\omega}$
j	Mechanical jerk $\ddot{\omega}$

B Implementation Mapping

`Source quantity`	Mathematical quantity
`RotorVelocityRefM`	$\omega^\star$
`RotorAccelerationRefM`	$\dot{\omega}^\star$
`RotorJerkRefM`	$\ddot{\omega}^\star$
`SpeedIntegralError`	$\int e_\omega dt$
`torqueFeedforward`	$J\dot{\omega}^\star + B\omega^\star + T_L$
`torqueCorrection`	ΔT
`iqRef`	$i_q^\star$
`iqRefDot`	$\dot{i}_q^\star$
`vdRef`	$v_d^\star$
`vqRef`	$v_q^\star$
`IdIntegralError`	$\int e_d dt$
`IqIntegralError`	$\int e_q dt$